

Curso desarrollo de aplicaciones con tecnologías web
Módulo formativo 2: Programación web en el entorno servidor

Unidad formativa 3: Desarrollo de aplicaciones web distribuidas

Tema 1: Arquitecturas distribuidas orientadas a servicios

Implementación de arquitecturas orientadas a servicios mediante tecnologías web

Implementación de la seguridad en arquitecturas orientadas a servicios

Directorios de servicios

Implementación de arquitecturas orientadas a servicios mediante tecnologías web

Las arquitecturas orientadas a servicios (SOA) pueden implementarse con tecnologías ampliamente usadas en la web, facilitando la comunicación entre sistemas heterogéneos a través de protocolos estándar. Estas tecnologías permiten construir, publicar, descubrir y consumir servicios.

1 Especificaciones de servicios web de uso común: SOAP, REST, etc.

Tecnología	Descripción	Características
SOAP (Simple Object Access Protocol)	Protocolo basado en XML para el intercambio estructurado de información entre servicios.	- Muy estructurado - Usa HTTP, SMTP u otros - Estándares como WSDL - Mejor para transacciones seguras y complejas
REST (Representational State Transfer)	Arquitectura que usa métodos HTTP (GET, POST, PUT, DELETE) para acceder a recursos representados en formatos como JSON o XML.	- Ligero y fácil de implementar - Usa URLs para identificar recursos - Muy usado en APIs modernas - Ideal para aplicaciones móviles o SPA
GraphQL	Alternativa moderna a REST que permite definir exactamente qué datos se desean en una sola petición.	- Más flexible - Reduce carga de datos innecesarios - Utiliza un único endpoint

Ejemplo de API REST (PHP usando file_get_contents):

php

```
<?php
// Obtener datos de la API de TheCatAPI
$response = file_get_contents("https://api.thecatapi.com/v1/images/search");
$data = json_decode($response, true);
echo "<img src='{ $data[0]['url']}' alt='Gato aleatorio'>";
?>
```

2 Lenguajes de definición de servicios: el estándar WSDL

WSDL (Web Services Description Language) es un lenguaje basado en XML que describe servicios web SOAP.

Especifica:

- Qué operaciones ofrece un servicio.
- Qué parámetros acepta.
- Qué datos devuelve.
- En qué formato y protocolo se comunica.

Ejemplo básico de definición en WSDL:

xml

```
<definitions name="ServicioUsuarios"
  targetNamespace="http://ejemplo.com/usuarios"
  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
  xmlns:tns="http://ejemplo.com/usuarios"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns="http://schemas.xmlsoap.org/wsdl/">

  <message name="GetUsuarioRequest">
    <part name="idUsuario" type="xsd:int"/>
  </message>
  <message name="GetUsuarioResponse">
    <part name="nombre" type="xsd:string"/>
  </message>

  <portType name="UsuarioPortType">
    <operation name="GetUsuario">
      <input message="tns:GetUsuarioRequest"/>
      <output message="tns:GetUsuarioResponse"/>
    </operation>
  </portType>
</definitions>
```

3 Estándares de seguridad en servicios web: WS-Security, SAML, XACML, etc.

Estándar	Función
WS-Security	Añade seguridad a los mensajes SOAP: firmas digitales, cifrado, tokens de autenticación.
SAML (Security Assertion Markup Language)	Permite el intercambio de información de autenticación y autorización entre dominios. Muy usado en SSO (Single Sign-On).
XACML (eXtensible Access Control Markup Language)	Lenguaje para definir políticas de control de acceso en servicios distribuidos. Muy útil para entornos complejos con reglas de permisos.

Ejemplo real de aplicación:

Una plataforma educativa que integra con Google (usando SAML) para que los usuarios puedan iniciar sesión con su cuenta institucional sin repetir credenciales.

Implementación de la seguridad en arquitecturas orientadas a servicios

La seguridad en arquitecturas orientadas a servicios (SOA) es fundamental, ya que múltiples servicios se comunican a través de la red y pueden estar expuestos a accesos no autorizados, modificaciones maliciosas o pérdida de datos. La implementación de la seguridad se basa en distintos niveles y estándares.

1 Conceptos básicos de criptografía

Criptografía: Ciencia que estudia técnicas para proteger información y comunicaciones. Se basa en transformar los datos de forma que solo usuarios autorizados puedan leerlos.

Tipos:

- **Criptografía simétrica:** La misma clave se usa para cifrar y descifrar.
- **Criptografía asimétrica:** Usa un par de claves (pública y privada).

Ejemplo con PHP:

php

```
$data = "Mensaje secreto";  
$clave = "123456";  
$cifrado = openssl_encrypt($data, "AES-128-ECB", $clave);  
echo $cifrado;
```

2 Tipos de criptografía

Tipo	Descripción	Uso común
Simétrica	Una sola clave secreta	Cifrado de datos en bases de datos o sesiones
Asimétrica	Clave pública y privada	Intercambio seguro de claves, firma digital
Hashing	Transformación unidireccional	Contraseñas (ej. password_hash en PHP)

3 Entidades certificadoras (CA)

Una **Autoridad Certificadora (CA)** es una entidad de confianza que emite certificados digitales para verificar la identidad de un servidor o usuario. Ejemplos: Let's Encrypt, DigiCert.

4 Certificados digitales. Características

Los certificados digitales:

- Autentican la identidad de un sitio o persona.
- Contienen información como nombre, clave pública, emisor y fecha de expiración.
- Se usan en HTTPS para cifrar conexiones.

Ejemplo:

Cuando visitas <https://midominio.com>, el navegador valida el certificado SSL emitido por una CA.

5 Identificación y firma digital mediante certificados digitales

La firma digital permite:

- Validar la integridad de un mensaje.
- Verificar la identidad del emisor.
- Prevenir la alteración de datos en tránsito.

PHP puede verificar firmas digitales usando extensiones como openssl.

6 Cifrado de datos

Además del uso de HTTPS, se pueden cifrar datos sensibles en aplicaciones:

php

```
$texto = "contraseña123";  
$clave = "mi_clave_segura";  
$cifrado = openssl_encrypt($texto, "AES-256-CBC", $clave, 0, "1234567890123456");
```

Nota: Es recomendable no implementar tu propio sistema de cifrado, sino utilizar herramientas seguras y actualizadas (como HTTPS, JWT o bibliotecas modernas).

Glosario de términos clave

Criptografía	Técnica para proteger datos mediante el cifrado.
Simétrica	Cifrado y descifrado con la misma clave.
Asimétrica	Cifrado con clave pública y descifrado con clave privada.
Certificado digital	Documento que valida la identidad de un sitio o usuario.
Autoridad Certificadora (CA)	Entidad que emite certificados digitales confiables.
HTTPS	Protocolo de navegación segura basado en TLS/SSL.
Firma digital	Código cifrado que valida identidad y contenido.
Hashing	Cálculo de una huella digital (hash) para proteger contraseñas.
JWT (JSON Web Token)	Método seguro para transmitir información como tokens en servicios web.

Directorios de servicios

1 Concepto de Directorio

Un **directorio de servicios** es un sistema que permite **registrar**, **buscar** y **acceder** a servicios disponibles en una arquitectura distribuida. Su función es ayudar a que distintas partes de una aplicación (por ejemplo, microservicios o APIs externas) puedan encontrarse entre sí sin tener que codificar sus direcciones manualmente.

2 Ventajas e Inconvenientes

Ventajas	Inconvenientes
Permiten el descubrimiento automático de servicios.	Pueden requerir configuración compleja.
Facilitan la escalabilidad de sistemas SOA.	Añaden una capa de complejidad en la infraestructura.
Posibilitan la reutilización de servicios ya existentes.	Algunos estándares (como UDDI) han caído en desuso.
Centralizan la gestión y documentación de servicios.	Requieren seguridad robusta para proteger el acceso a los servicios.

3 Directorios Distribuidos

Los **directorios distribuidos** permiten registrar y consultar servicios desde múltiples ubicaciones de forma sincronizada. Funcionan como una red de registros interconectados y permiten mayor escalabilidad y disponibilidad.

- Son útiles en sistemas grandes o geográficamente distribuidos.
- Aseguran que los servicios puedan ser descubiertos en cualquier nodo de la red.

Ejemplo práctico:

Empresas que ofrecen microservicios desde distintas ubicaciones o servidores distribuidos pueden usar un directorio distribuido para que las apps cliente siempre encuentren el servicio más cercano o disponible.

4 Estándares sobre Directorios de Servicios: UDDI (obsoleto en la web actual)

El estándar **UDDI** tenía como objetivo servir de “páginas amarillas” para servicios web basados en SOAP. Permite publicar, descubrir y describir servicios usando **WSDL**.

Sin embargo, **UDDI ha quedado en desuso**, y su complejidad no encaja bien con las arquitecturas web actuales, que usan principalmente **REST** y **JSON**.

5 Alternativas prácticas actuales en tecnologías web (PHP, HTML, SQL, JS)

-Documentación de APIs tipo REST con archivos JSON

En vez de usar un directorio externo, los servicios se describen en archivos `openapi.json` o `swagger.json`, que sirven como contrato técnico.

Ejemplo básico (api.json):

json

```
{
  "openapi": "3.0.0",
  "info": {
    "title": "API de Productos",
    "version": "1.0.0"
  },
  "paths": {
    "/productos": {
      "get": {
        "summary": "Devuelve la lista de productos",
        "responses": {
          "200": {
            "description": "Lista de productos"
          }
        }
      }
    }
  }
}
```

Puedes **leer este archivo desde PHP** y usarlo para generar automáticamente una lista navegable de endpoints en tu aplicación.

-Página HTML como “directorio de servicios” simple

Una web propia en **HTML/PHP** puede actuar como directorio manual de servicios si trabajamos con APIs propias o externas.

 Ejemplo (PHP):

```
$servicios = [  
    ['nombre' => 'API de productos', 'url' => 'https://api.miapp.com/productos'],  
    ['nombre' => 'API de usuarios', 'url' => 'https://api.miapp.com/usuarios'],  
];  
  
foreach ($servicios as $servicio) {  
    echo "<li><a href='{${servicio['url']}'}'>{${servicio['nombre']}</a></li>";  
}
```

Este archivo se puede consumir con JavaScript (fetch) o PHP (file_get_contents() y json_decode()).

-JSON como base de un pequeño registro de servicios

Puedes usar un **archivo JSON en local** (o generado por PHP) como catálogo de tus servicios:

json

```
[
  {
    "nombre": "API Productos",
    "endpoint": "/api/productos",
    "metodo": "GET"
  },
  {
    "nombre": "API Usuarios",
    "endpoint": "/api/usuarios",
    "metodo": "GET"
  }
]
```

Ejercicio práctico1: Consumo y análisis de un servicio web REST

Objetivo: Comprender cómo se integran servicios distribuidos en una aplicación web usando tecnologías como REST y JSON.

Enunciado:

Vas a integrar un servicio web REST público en una página PHP. Para ello:

1. Utiliza la API pública de OpenWeatherMap o TheCatAPI (elige una).
2. Realiza una petición desde PHP para obtener información.
3. Muestra los datos recibidos en una página web (por ejemplo: nombre del lugar, temperatura o una imagen de gato aleatorio).
4. Da formato al resultado usando Bootstrap para que se vea atractivo.
5. Explica qué tipo de arquitectura tiene la API (REST o SOAP), si se basa en recursos o mensajes, y qué formato de datos devuelve (JSON, XML...).

Requisitos:

- Conectar mediante `file_get_contents()` o cURL desde PHP.
- Decodificar la respuesta (si es JSON).
- Mostrar al menos un dato dinámico (imagen, texto, número...).
- Añadir una pequeña caja con Bootstrap que contenga el contenido.
- Documentar en un comentario qué características del servicio se han identificado (formato, arquitectura, método HTTP...).

Resultado esperado:

Una página en PHP que muestre dinámicamente información de un servicio REST (por ejemplo, una tarjeta con el clima actual de una ciudad, o una imagen de un gato con su ID), usando Bootstrap para el diseño y comentarios explicativos sobre el servicio.

Ejercicio práctico 2

Formulario Seguro con Cifrado y Validación

Descripción:

Desarrolla un formulario que recoja el nombre y el correo de un usuario y almacene los datos en un fichero de forma cifrada, aplicando además buenas prácticas de seguridad como la validación y el uso de hash.

Requisitos:

1. Crear un formulario HTML con los siguientes campos:
 - Nombre (<input type="text">)
 - Email (<input type="email">)
 - Botón de envío
2. Validar los datos en PHP:
 - Comprobar que el nombre no esté vacío.
 - Comprobar que el email tenga un formato válido.
3. Cifrar el nombre con AES antes de almacenarlo.
4. Almacenar el email usando hash('sha256', ...) como medida de privacidad.
5. Guardar los datos cifrados en un archivo .txt, con el siguiente formato:
nombre_cifrado | email_hash | fecha
6. Mostrar un mensaje al usuario indicando que el formulario fue procesado correctamente.

Consejos técnicos:

- Puedes usar openssl_encrypt() para cifrar el nombre.
- Usa date('Y-m-d H:i:s') para almacenar la fecha del envío.
- Guarda los datos en un archivo datos.txt usando file_put_contents() con el flag FILE_APPEND.

Resultado esperado (visual y técnico):

Un pequeño formulario de contacto funcional, seguro y que genera un fichero como este:

3ea4a1efde8c52... | 6a3dc9f64e... | 2025-04-10 11:37:00

Ejercicio práctico 3

Objetivo: Crear un “directorio de servicios” en PHP.

Instrucciones:

1. Crea un archivo servicios.json que contenga al menos 3 servicios ficticios.
2. Haz una página directorio.php que lea ese JSON y lo muestre como lista navegable con Bootstrap.
3. Opcional: añade un formulario para añadir nuevos servicios al archivo.